
Scientific Python for Machine Learning

Refresher Module 1 — Python, NumPy and Pandas

the data model · the array model · broadcasting · the tabular model · arrays to tensors

Computing, Robotics & Bionics

3rd-year Bioengineering — companion to A. Géron, *Hands-On Machine Learning with Scikit-Learn and PyTorch*

*This is the first of four refresher modules that prepare you to step straight into Géron's book. It supplies the conceptual background you need before the code: how Python represents data, why the NumPy array (not the Python list) is the unit of scientific computing, the broadcasting rules that make array code both short and fast, the labelled-table model behind Pandas, and the small step from a NumPy array to a PyTorch tensor. It is **self-contained**: together with its four companion Colab notebooks — a bioengineering-adapted superset of Géron's `tools_numpy`, `tools_pandas` and `tools_matplotlib` material — it is your complete reference, so you never have to read two versions side by side. The notebooks put every idea to work on biomedical data (diagnostic tables and physiological signals). Each section ends with exercises.*

Prerequisites: high-school algebra and some prior exposure to programming. Software: a web browser and a Google account (we run everything in Google Colab; nothing to install).

Contents

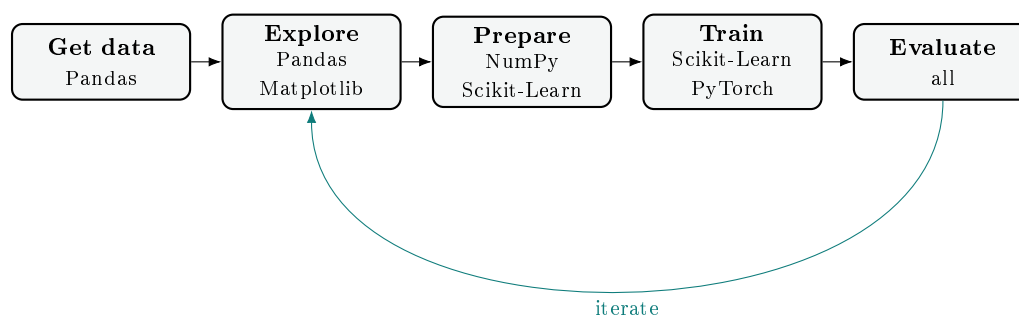
1	The Scientific Python Stack and the ML Workflow	2
2	Python’s Data Model: the Parts the Book Assumes	3
2.1	Objects and names	3
2.2	The built-in containers	3
2.3	Iteration, comprehensions and functions	3
3	The Array Model: NumPy	4
3.1	Why not just use lists?	4
3.2	Vectorisation: say what, not how	4
3.3	Boolean masks and fancy indexing	5
3.4	Broadcasting: arrays of different shapes	5
3.5	Views versus copies (the aliasing trap)	6
4	The Tabular Model: Pandas	6
5	From Arrays to Tensors: the PyTorch Bridge	7
6	A Worked Thread: a Biomedical Table, End to End	8
7	The Lab, and Where This Leads in Géron	8

1 The Scientific Python Stack and the ML Workflow

Machine learning in Python is built on a small, stable stack of libraries, each owning one job. You will use all of them in Géron’s book, so it pays to know where each fits before writing a line of code.

Library	Its one job (and when it appears in the book)
NumPy	the numerical array — the unit of all scientific computing in Python; everything else is built on it
Pandas	labelled tables (rows = samples, columns = features); loading, cleaning and exploring data (the end-to-end project)
Matplotlib	plotting; you visualise data and results constantly
Scikit-Learn	classical machine-learning models with one uniform interface (<code>fit</code> , <code>predict</code> , <code>transform</code>)
PyTorch	tensors, automatic differentiation and neural networks; the deep-learning half of the book

The shape of almost every ML project — and the spine of Géron’s Chapter 2 — is the same pipeline, and each stage is owned by one of these libraries:



Key idea

Everything in this course runs in **Google Colab**: a free, browser-based Jupyter notebook with NumPy, Pandas, Matplotlib, Scikit-Learn and PyTorch already installed, and a free GPU available from the *Runtime* → *Change runtime type* menu. There is nothing to install on your laptop; you only need a browser and a Google account. A *cell* holds code or text; **Shift+Enter** runs the current cell.

How this module relates to Géron’s book. Géron’s book assumes you can already read scientific Python, and ships reference notebooks (`tools_numpy`, `tools_pandas`, `tools_matplotlib`) for those who want them. *You will not need to go back and forth.* This module and its four companion notebooks are **self-contained**: together they are your complete, bioengineering-adapted reference for Python, NumPy, Pandas and plotting — a superset of that material, re-ordered around machine learning and worked on biomedical data. Section and notebook numbers point forward to the exact chapters of the book they prepare you for, so when you open Géron you step straight in.

2 Python’s Data Model: the Parts the Book Assumes

Géron assumes you can read and write idiomatic Python. This section is the minimum model of how Python represents and moves data — not a full language tutorial, but the pieces that trip people up when they reach real ML code.

2.1 Objects and names

In Python *everything is an object*, and a variable is just a **name** bound to an object, like a luggage tag — not a box holding a value. Assignment binds a name; it never copies the object. Two names can tag the same object, which matters the moment objects are *mutable*.

```
1 a = [1, 2, 3]           # 'a' tags a list object
2 b = a                  # 'b' tags the SAME object (no copy!)
3 b.append(4)
4 print(a)               # -> [1, 2, 3, 4] ... 'a' changed too
```

Definition

An object is **mutable** if its contents can change in place (`list`, `dict`, `set`, NumPy arrays, Pandas frames) and **immutable** if they cannot (`int`, `float`, `str`, `tuple`, `bool`). Binding a new name to a mutable object creates an *alias*, not a copy — the single most common source of silent bugs in data code. To copy, ask explicitly (`list(a)`, `a.copy()`, `arr.copy()`).

2.2 The built-in containers

Four containers carry almost all data before it reaches NumPy:

Type	Literal	Use it for
<code>list</code>	<code>[1, 2, 3]</code>	an ordered, mutable sequence; grows and shrinks
<code>tuple</code>	<code>(1, 2, 3)</code>	a fixed, immutable record; e.g. an array <i>shape</i>
<code>dict</code>	<code>{"a": 1}</code>	key → value lookup; e.g. a row, or model hyper-parameters
<code>set</code>	<code>{1, 2, 3}</code>	unordered unique items; membership and de-duplication

Sequences are indexed from 0 and support **slicing** `seq[start:stop:step]`, where `stop` is excluded and negative indices count from the end. The same slice syntax reappears, generalised to many dimensions, in NumPy and Pandas — so it is worth being fluent: `x[1:4]`, `x[:3]`, `x[::-1]` (reversed), `x[-1]` (last).

2.3 Iteration, comprehensions and functions

Python loops over any **iterable** (lists, strings, dict keys, files, ranges). The Pythonic way to build a new sequence from an old one is a **comprehension**, which you will read constantly in ML code:

```
1 squares = [x**2 for x in range(10)]           # list comprehension
2 evens    = [x for x in range(10) if x % 2 == 0] # with a filter
3 name2id  = {name: i for i, name in enumerate(labels)} # dict comprehension
```

Functions are themselves objects: they can be stored in variables, passed as arguments, and given default and keyword arguments. A short anonymous function is a **lambda**. This first-class treatment is why Scikit-Learn and PyTorch can take a function (a metric, a transform, an activation) as an argument.

```

1 def standardize(x, mean=0.0, std=1.0):    # defaults
2     return (x - mean) / std
3
4 f = standardize                          # functions are objects
5 relu = lambda z: max(0.0, z)             # a tiny anonymous function

```

Key idea

You will *read* far more classes than you write. Scikit-Learn and PyTorch are object-oriented: a model is an *object* with *attributes* (data it stores, e.g. `model.coef_`) and *methods* (things it does, e.g. `model.fit(X, y)`). You rarely need to define your own class to use the book — until PyTorch, where a network is written as a small class subclassing `nn.Module`. Knowing how to read `object.attribute` and `object.method(args)` is enough to start.

Exercises

1. Predict the output: `a=[1,2]; b=a; b+=[3]; print(a)`. Then predict it for `a=(1,2); b=a; b+=(3,); print(a)`. Explain the difference using mutability.
2. Write a list comprehension that returns the lengths of the words in a list `words`, keeping only words longer than three letters.
3. Given `x=list(range(10))`, write slices that produce: the first five elements; the last three; every second element; the list reversed.
4. Explain, in one sentence each, the difference between an object's *attribute* and its *method*, with a Scikit-Learn example of each.

3 The Array Model: NumPy

3.1 Why not just use lists?

A Python `list` can hold anything, but that flexibility is expensive: each element is a separate object scattered in memory, and a loop over a list runs in the (slow) Python interpreter. A **NumPy array** (`ndarray`) is the opposite: a single block of memory holding elements that all share one numeric type, so operations run as compiled C loops over contiguous data — often 10–100× faster, and far more compact.

Definition

An `ndarray` is a typed, fixed-size, N -dimensional grid of numbers. It is described by three things: its **shape** (a tuple giving the length along each axis, e.g. `(100, 3)` for 100 samples of 3 features), its **dtype** (the element type, e.g. `float64`, `int32`, `uint8`), and the **data** itself in one contiguous buffer.

The shape is the single most important property of an array; most bugs in ML code are shape mismatches. Read `X.shape` constantly. By convention a design matrix is shaped `(n_samples, n_features)` — rows are samples, columns are features — and this convention is shared by Scikit-Learn and (with a batch axis) PyTorch.

3.2 Vectorisation: say what, not how

The central idiom of scientific Python is to express an operation on a *whole array at once* rather than element by element. This is **vectorisation**: it is shorter, reads like the maths, and runs at compiled speed.

```

1 import numpy as np
2 v = np.array([60.0, 72.0, 80.0])      # heart rates (bpm)
3 # normalise to a 0-1 range -- one expression, no loop:
4 vn = (v - v.min()) / (v.max() - v.min())
5 # elementwise maths applies to the whole array:
6 rms = np.sqrt(np.mean(v**2))

```

Key idea

Rule of thumb: if you are writing a **for loop** over array elements, there is **almost certainly a vectorised version** that is shorter and much faster. Reach for whole-array expressions, reductions (**sum**, **mean**, **std**, **max**), and boolean masks instead of loops.

Operations along a chosen axis are controlled by the **axis** argument, and this is worth picturing concretely. For **X** of shape **(n_samples, n_features)**: **X.mean(axis=0)** collapses the rows and returns one mean *per feature* (length **n_features**); **X.mean(axis=1)** collapses the columns and returns one mean *per sample*. “The axis you name is the axis that disappears.”

3.3 Boolean masks and fancy indexing

A comparison on an array yields a boolean array, which can then *select* elements — the workhorse of data filtering, and far cleaner than a loop with an **if**.

```

1 ages = np.array([34, 71, 19, 58, 65])
2 mask = ages >= 65      # array([False, True, False, False, True])
3 seniors = ages[mask]   # array([71, 65])
4 ages[ages < 18] = 18   # clip in place with a mask

```

3.4 Broadcasting: arrays of different shapes

NumPy lets you combine arrays of *different but compatible* shapes without writing loops or copying data, by **broadcasting** the smaller array across the larger. This is how you add a per-feature vector to every row of a matrix, or scale a colour image by three channel weights.

Definition

To broadcast two shapes, NumPy aligns them **from the right** and, for each axis, requires the lengths to be *equal* or one of them to be *1* (a length-1 axis is stretched to match). A missing leading axis counts as 1.

Centring a data matrix

Let **X** have shape **(100, 3)** (100 samples, 3 features) and let the per-feature mean **mu** = **X.mean(axis=0)** have shape **(3,)**. Then **X - mu** broadcasts:

$$\underbrace{(100, 3)}_X - \underbrace{(3,)}_{\text{mu}} \longrightarrow (100, 3) - (1, 3) \longrightarrow (100, 3),$$

subtracting the same three means from every one of the 100 rows — with no loop and no explicit copy of **mu**. This one line is feature *centring*, the first step of standardisation and of PCA (Module 2).

When the shapes are *not* compatible (e.g. **(100, 3)** with **(100,)**) NumPy raises a **ValueError**; the fix is usually to insert a length-1 axis with **mu[:, None]** so the alignment works. Reading and fixing such shape errors is a core daily skill.

3.5 Views versus copies (the aliasing trap)

For speed, a slice of an array is usually a **view** — a window onto the *same* memory, not a copy. Writing through a view changes the original. This is efficient but surprising, and is the array-level version of the name-aliasing we saw for lists.

```
1 A = np.arange(12).reshape(3, 4)
2 row = A[0]           # a VIEW of the first row
3 row[:] = 0           # also zeroes that row inside A!
4 B = A[0].copy()      # an explicit copy; safe to modify
```

Key idea

When you intend to modify a sub-array without touching the original, take an explicit `.copy()`. When you merely read, the view is a free performance win. Knowing which you have prevents a whole class of “my data changed by itself” bugs.

Exercises

1. For `X` of shape `(50, 4)`, what shape does `X.sum(axis=0)` have? `X.sum(axis=1)`? State which axis “disappears” in each.
2. Can shapes `(8, 1, 6)` and `(7, 1)` be broadcast together? If so, give the result shape; if not, say why (align from the right).
3. Using a boolean mask, write one line that sets every negative value of an array `z` to zero (this is the ReLU function).
4. Explain why `b = A[:, 0]; b[0] = 99` can change `A`, and how to avoid it when unwanted.

4 The Tabular Model: Pandas

Raw data rarely arrives as a clean numeric array; it arrives as a table with named columns, mixed types and missing entries — a CSV from an instrument, a clinical export, a spreadsheet. **Pandas** is the tool for that table, and it is where every ML project begins.

Definition

A **Series** is a one-dimensional labelled array (values plus an *index*). A **DataFrame** is a two-dimensional table: an ordered set of columns, each a Series, sharing one row index. Think “NumPy array with named rows and columns, whose columns may have different types.”

Two ideas make Pandas more than a spreadsheet. First, the **index**: rows and columns carry labels, and operations *align on those labels* automatically, so adding two Series matches entries by index rather than by position. Second, **split–apply–combine**: `groupby` splits rows into groups, applies a function to each, and combines the results — the natural way to compute, say, a mean biomarker per diagnosis.

```
1 import pandas as pd
2 df = pd.read_csv("patients.csv")      # a DataFrame
3 df.head(); df.info(); df.describe()   # always look first
4 df["bmi"] = df["mass"] / df["height"]**2 # new column (vectorised)
5 adults = df[df["age"] >= 18]           # boolean row filter
6 df.groupby("diagnosis")["glucose"].mean() # split-apply-combine
```

Selecting rows and columns. Pandas separates label-based selection (`df.loc[rows, cols]`) from position-based selection (`df.iloc[i, j]`). Mixing them up is a common early error; the rule is simple — `loc` speaks *labels*, `iloc` speaks *integer positions*.

Missing data. Real datasets have holes, represented as `NaN`. Pandas gives you `df.isna()` to find them and `dropna/fillna` to handle them; Géron's Chapter 2 treats imputation properly, but you must first be able to *see* the missingness. Tidy, complete numeric tables are what you hand to Scikit-Learn — typically as `df.values` or `df.to_numpy()`, which returns the underlying NumPy array, closing the loop back to Section 3.

Exercises

1. State the difference between `df.loc[0]` and `df.iloc[0]` for a DataFrame whose index labels are not 0, 1, 2,
2. Write one line that returns the mean `glucose` for each `diagnosis` group, sorted from highest to lowest.
3. Write a boolean filter selecting rows where `age` is between 40 and 60 inclusive *and* `glucose` exceeds 126.
4. How would you find how many missing values each column has? (One expression.)

5 From Arrays to Tensors: the PyTorch Bridge

The deep-learning half of Géron's book uses **PyTorch**, whose core object is the `tensor`. The good news: a tensor is, for our purposes, a NumPy array with three extra abilities. Everything you learned about shape, dtype, indexing and broadcasting carries over unchanged.

A tensor adds... ... which a NumPy array lacks

a device	it can live on a GPU, where large matrix maths runs far faster
autograd	it can record the operations done to it and compute gradients automatically (Module 3)
batch-first shapes	deep learning adds a leading <i>batch</i> axis: <code>(batch, features)</code> , or <code>(batch, channels, height, width)</code> for images

Conversion is direct, and on the CPU an array and a tensor can even *share* memory, so no data is copied:

```
1 import torch
2 X = np.random.rand(100, 3).astype("float32")
3 t = torch.from_numpy(X)      # array -> tensor (shares memory on CPU)
4 back = t.numpy()             # tensor -> array
5 t = t.to("cuda")             # move to GPU if available (Colab: free GPU)
6 t.shape, t.dtype             # same mental model: shape and dtype
```

Key idea

Two practical gotchas. First, deep learning defaults to `float32` (not NumPy's `float64`); cast with `.astype("float32")` or `.float()` to avoid type errors. Second, the leading **batch axis** is everywhere in PyTorch — a single image of shape `(3, 224, 224)` is fed to a network as a batch of one, shape `(1, 3, 224, 224)`. We meet this again in the image-processing module.

6 A Worked Thread: a Biomedical Table, End to End

To see the pieces working together, here is the conceptual walk-through you will carry out fully in the Colab notebook, on a real diagnostic dataset (the Wisconsin breast-cancer data that ships with Scikit-Learn — no download needed).

1. **Load** the data into a DataFrame; inspect with `head`, `info`, `describe`; confirm shape (569, 30) — 569 samples, 30 features — and check for missing values.
2. **Explore**: group by diagnosis (malignant/benign) and compare a feature such as `mean radius`; plot a histogram per class with Matplotlib to see the separation.
3. **Prepare**: pull the numeric matrix with `to_numpy()`; centre and scale each feature using broadcasting, $(X - X.mean(0)) / X.std(0)$ — exactly the standardisation Scikit-Learn's `StandardScaler` performs.
4. **Bridge**: convert the prepared `float32` matrix to a PyTorch tensor and confirm its shape and device, ready for the models of later chapters.

Notice that every stage used one idea from these notes: the tabular model, the split–apply–combine, vectorised standardisation by broadcasting, and the array-to-tensor bridge. That is the whole pipeline of Figure in Section 1, in miniature.

7 The Lab, and Where This Leads in Géron

Key idea

Companion Colab notebooks. After the theory, the session moves to Google Colab. Module 1 ships *four* self-contained notebooks — together your complete, biomedical reference, with no need to consult any other material:

- `m1_python_basics` — the Python fundamentals the book assumes;
- `m1_numpy` — arrays end to end, with a synthetic-ECG and PhysioNet thread;
- `m1_pandas` — labelled tables, on the breast-cancer and DepMap/CCLE data;
- `m1_matplotlib` — plotting, including `imshow` (which sets up the image module).

Each is a superset of Géron's corresponding reference notebook, re-ordered around machine learning and worked on biomedical data, and each ends its sections with exercises and worked solutions.

Where this unlocks the book. With this module you are ready for **Chapter 2** (the end-to-end project: Pandas loading and exploration, NumPy preparation) and **Chapter 3** (classification, where the design matrix and label vector are exactly the `(n_samples, n_features)` arrays met here). The array and broadcasting models return in *every* later chapter; the tensor bridge is your on-ramp to **Chapter 10** and the deep-learning half.

The one-paragraph summary

A name is a tag, not a box; mutable objects can be aliased, so copy on purpose. The `ndarray` — typed, contiguous, shape-described — is the unit of computation; prefer whole-array (vectorised) expressions, reductions over a named `axis`, boolean masks, and broadcasting over loops, and mind views versus copies. Pandas is a labelled table with an aligning index and split–apply–combine; it hands a clean NumPy matrix to the models. A PyTorch tensor is that same array, now able to live on a GPU and carry gradients. Master these and Géron's code reads like prose.